

SymPy Bug Report

1 Bug 34: Branch Cut Error in Radical Solving

1.1 Minimal Reproducer

```
from sympy import Eq, S, sqrt, solveset, symbols

x = symbols("x")
sol = solveset(Eq(sqrt(1/x), 1/sqrt(x)), x, domain=S.Complexes)

print("solution:", sol)
print("contains -1:", sol.contains(S.NegativeOne))
print("lhs at -1:", sqrt(1/x).subs(x, -1))
print("rhs at -1:", (1/sqrt(x)).subs(x, -1))
print("residual at -1:", (sqrt(1/x) - 1/sqrt(x)).subs(x, -1))
```

1.2 Observed Behavior

```
solution: Complement(Complexes, {0})
contains -1: True
lhs at -1: I
rhs at -1: -I
residual at -1: 2*I
```

SymPy reports all nonzero complex numbers as solutions, so the returned set includes $x = -1$. Direct substitution into the original equation gives a nonzero residual, $2i$, which proves that the reported set contains invalid solutions.

1.3 Expected Behavior

The returned complex-domain solution set should exclude 0 and the negative real axis. In particular, `contains(-1)` should be false. A mathematically correct description is

$$\mathbb{C} \setminus (-\infty, 0],$$

where 0 is excluded because the original expression is undefined and the negative real axis is excluded by the principal-square-root branch.

1.4 Mathematical Explanation

SymPy's $\sqrt{\cdot}$ denotes the principal square root. For $x = -1$,

$$\sqrt{1/x} = \sqrt{-1} = i, \quad \frac{1}{\sqrt{x}} = \frac{1}{\sqrt{-1}} = \frac{1}{i} = -i.$$

Thus

$$\sqrt{1/x} - \frac{1}{\sqrt{x}} = 2i \quad \text{at} \quad x = -1,$$

so $x = -1$ is not a solution.

More generally, write a nonzero complex number as $x = re^{i\theta}$ with the principal argument $\theta \in (-\pi, \pi]$. For $\theta \in (-\pi, \pi)$, the reciprocal has principal argument $-\theta$, and the two sides agree. On the negative real axis, however, $\theta = \pi$ and $1/x$ is also on the negative real axis with principal argument π . The principal square root therefore picks the opposite sign from $1/\sqrt{x}$. This is not a harmless alternate representation: SymPy's own substitution shows distinct values i and $-i$.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant source location as `sympy/solvers/solveset.py`, in the helper `_solve_radical` around lines 1090–1139. The traced call path is

```
solveset → _solveset → unrad → _solve_radical.
```

For this equation, `solveset` first rewrites the input as $\sqrt{1/x} - 1/\sqrt{x}$. After denominator clearing, `unrad` reduces the numerator $\sqrt{x}\sqrt{1/x} - 1$ to an identity, recorded as $(0, [])$. The radical-free problem is then treated as true for all complex numbers except denominator singularities, producing $\mathbb{C} \setminus \{0\}$.

The source-level issue is that `_solve_radical` validates finite candidate sets with `checksol`, but its infinite-set handling does not recheck the original branch-sensitive equation over the broad result. The diagnosis highlights this pattern:

```
elif isinstance(solutions, Complement):
    A, B = solutions.args
    return Complement(check_set(A), B)
...
else:
    return solutions
```

In this case, `check_set(S.Complexes)` remains `S.Complexes`, and only 0 is subtracted as a denominator singularity. The negative real axis survives even though the original principal-square-root equation rejects it. A plausible fix direction supported by the diagnosis is to preserve an explicit condition or solve the missing branch constraints whenever radical removal yields broad infinite sets such as $\mathbb{C}$, intervals, or complements.

1.6 Additional Failing Instances

The independent verification checks the representative value $x = -1$ and five additional negative real values. For each case, SymPy's returned set claims membership while direct principal-square-root evaluation gives a nonzero residual. The expected behavior is the same across the family because every negative real value lies on the principal square-root branch cut, where $\sqrt{1/x}$ and $1/\sqrt{x}$ choose opposite signs.

Input / Expression	SymPy behavior	Expected behavior
$x = -1$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains -1 ; residual $2i$	Exclude -1
$x = -2$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains -2 ; residual nonzero	Exclude -2
$x = -3$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains -3 ; residual nonzero	Exclude -3
$x = -1/2$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains $-1/2$; residual nonzero	Exclude $-1/2$
$x = -5$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains -5 ; residual nonzero	Exclude -5
$x = -7/3$ in $\sqrt{1/x} = 1/\sqrt{x}$	Returned set contains $-7/3$; residual nonzero	Exclude $-7/3$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest related public material concerns the general principal-square-root branch-cut family: SymPy documentation explains that $\sqrt{\cdot}$ is principal, SymPy simplification guidance warns that power identities need assumptions, and public mathematical discussions cover why square-root multiplicativity or reciprocal identities fail on branch cuts. The available evidence did not identify a clear public duplicate for this exact `solveset` call returning $\mathbb{C} \setminus \{0\}$ and including $x = -1$. The case is therefore individually actionable as a concrete solver regression test and fix target.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, Rational, S, simplify, sqrt, solveset, symbols

@pytest.mark.parametrize(
    "bad_value",
    [
        S.NegativeOne,
        S(-2),
        S(-3),
        Rational(-1, 2),
        S(-5),
        Rational(-7, 3),
    ],
)
def test_solveset_sqrt_reciprocal_excludes_negative_real_axis(bad_value):
    x = symbols("x")
    lhs = sqrt(1/x)
    rhs = 1/sqrt(x)

    sol = solveset(Eq(lhs, rhs), x, domain=S.Complexes)
```

```
assert sol.contains(bad_value) is S.false
assert simplify((lhs - rhs).subs(x, bad_value)) != 0
```